FICHE MÉMO

Bases du développement agentique

Les notions, les prompts et les commandes du Module 1, dans l'ordre du deck : cadrer, générer (greenfield), faire évoluer (brownfield), brancher des outils (MCP).

L'AI-Driven Development

Un mode de travail où le développeur s'appuie sur des **agents IA** (Claude Code, GitHub Copilot…) plutôt que d'écrire chaque ligne à la main. Le code généré doit être **aussi proche que possible** de celui qu'on aurait écrit : des guidelines, les meilleurs modèles, une relecture systématique. Le développeur **reste responsable** du code généré.

	Vibe coding	AI-driven development
Public	Product Managers, designers UX/UI…	Ingénieurs logiciel
Code	Caché par défaut	L'IDE reste l'interface ; le dev maîtrise le code
Portée	Prototypage rapide d'applis neuves	Toute appli (legacy, neuve, petite ou grosse…)
Outils	Replit, Lovable, Bolt	Claude Code, GitHub Copilot, Cursor, Windsurf, Antigravity

Les outils **convergent** : les mêmes techniques s'appliquent partout. Les pas-à-pas du module s'appuient sur Claude Code ; une version symétrique existe pour GitHub Copilot.

> L'IA amplifie le développeur, elle ne le remplace pas. C'est vous qui pilotez.

Les 4 réflexes

1. **Cadrer** : des instructions custom (`CLAUDE.md`).
2. **Planifier** : le mode plan avant de générer.
3. **Outiller** : skills, agents, MCP.
4. **Relire** : les tests, le diff.

Planifier, relire, gérer le contexte

Planifier avant de coder : définir le problème, le découper, identifier les contraintes (API, sécurité, performance), prototyper mentalement, puis générer.

Relire le code généré comme celui d'un contributeur inconnu :

Axe	Question
Correction	Résout-il vraiment le problème ?

Axe	Question
Cas limites	Entrée vide, `null`, grandes valeurs ?
Sécurité	Entrées validées ? autorisations correctes ?
Performance	N+1, boucles infinies, fuites mémoire ?
Lisibilité	Compréhensible dans 6 mois par l'équipe ?
Dépendances	Les imports existent-ils vraiment et sont-ils à jour ?

```text
Relis le code que tu viens d'écrire : cas limites, sécurité, perf.
```

Gérer le contexte : la context window est limitée. Être sélectif, résumer, mentionner les fichiers utiles, découper en conversations courtes, garder le contexte persistant dans des fichiers d'instructions.

```text
Corrige la fonction calculateDiscount dans pricing.service.ts.
Elle reçoit un Cart et un PromoCode, et ignore aujourd'hui
le montant minimum d'achat.
```

Claude Code : modes de permission et mode plan

On bascule avec **Shift+Tab**.

Mode	Ce qui demande une validation
`default`	Tout (fichiers, commandes)
`acceptEdits`	Commandes seulement : les éditions de fichiers sont auto-acceptées
`plan`	Lecture seule : propose un plan, aucune édition
`auto` (Teams / Enterprise)	Un classifieur vérifie chaque action, interruptions minimales

> Commencez en **plan**, validez, puis passez en **acceptEdits** pour exécuter. Toujours le mode plan pour les gros changements multi-fichiers.

`CLAUDE.md` : les instructions custom

Chargé **au début de chaque session**. Il prescrit les **pratiques de code** (stack, conventions, commandes) et des **informations générales** sur l'appli.

- Rester **sous ~200 lignes** ; au-delà, l'adhérence chute.
- Des règles **vérifiables** : « Lance `npm test` avant chaque commit » plutôt que « Teste tes changements ».
- `/init` génère un `CLAUDE.md` de départ en analysant le code.
- Portées : `./CLAUDE.md` (projet, versionné), `~/.claude/CLAUDE.md` (perso), niveau organisation (géré par la DSI).

Greenfield : le workflow

1. **Définir** : une spécification claire dans un fichier markdown externe.
2. **Architecturer** en mode plan : structure, schéma, endpoints, décisions et compromis, dans `architecture.md`, sans code.
3. **Scaffolder** : structure, configuration, dépendances, un premier composant end-to-end, le fichier d'instructions IA, sans logique métier.
4. **Implémenter feature par feature** : planifier, implémenter, relire, refactorer, ajouter la doc IA manquante, mettre à jour la doc projet, feature suivante.

```text
À partir de cette spécification, propose :
1. La structure de dossiers/modules d'un projet NestJS
2. Le schéma de base (entités et relations)
3. Les endpoints de l'API (design REST)
4. Les décisions d'architecture clés et leurs compromis
N'écris aucun code. Rédige ta proposition dans architecture.md.
```

Documentation-Driven Development : faire écrire la doc de la feature (besoin, implémentation cible, questions ouvertes) avant le code.

Pièges : sur-scaffolding, tests sautés, dérive d'architecture, dépendances hallucinées, logique métier manquante. Contre-mesures : lire chaque fichier généré, lancer les tests en continu, vérifier que les dépendances existent, tester avec des données réalistes.

Skills et agents

Une **skill** emballe un savoir-faire reproductible : un `SKILL.md` (mode d'emploi) + des fichiers de support, chargé automatiquement quand sa description correspond à la tâche. `/skills` liste les skills disponibles.

```markdown
---
name: frontend
description: À utiliser pour les tâches de développement frontend.
---

- Charge d'abord architecture-frontend.md
- Crée un plan avant d'implémenter
- Exécute le plan étape par étape, en attendant ma validation
```

Skills de convention : une skill atomique par tâche récurrente (`new-endpoint` , `new-service` , `add-tests` , `new-migration`). Une skill qui couvre tout est ignorée.

Sous-agents : instances Claude isolées, avec leur propre context window et leur jeu de tools. Intégrés : Explore, Plan, general-purpose. Custom : `.claude/agents/<nom>.md` (tools, modèle, system prompt, mémoire persistante). Seul le résumé revient dans la conversation principale.

Brownfield : faire évoluer un code existant

- **Comprendre** : demander l'architecture à partir du point d'entrée, ou le rôle d'un fichier.
- **Fichiers de contexte** : `/init` , puis relire de façon critique (concepts métier, pièges et invariants, patterns implicites). Sur un gros code, des fichiers d'instructions **par module**, qui évoluent avec le code.

- **Respecter l'existant** : faire regarder les patterns en place avant de proposer ; « n'introduis aucun nouveau pattern ni dépendance ».
- **Tests sur du legacy** : la tâche à plus forte valeur ; filet de sécurité, documentation du comportement.
- **Refactorer en sécurité** : identifier, tester, planifier, transformer, vérifier, relire. Ne jamais sauter les tests ni la vérification. Nommer le pattern précis (Extract Function, Strategy, Introduce Repository…).
- **Tests de caractérisation** : capturer le comportement actuel avant un gros refactor, pour détecter tout changement.

```text
Avant de refactorer InvoiceCalculator, génère des tests qui capturent
le comportement ACTUEL, peu importe qu'il soit correct : je veux savoir
si je change quelque chose par accident.
```

MCP : brancher des outils tiers

Le **Model Context Protocol** est un standard ouvert (Anthropic, 2024) : une interface standard entre une IA et des outils externes, en client-serveur, avec des **resources**, des **tools** et des **prompts**. L'« USB des outils IA ».

Serveur MCP	Ce qu'il débloque
GitHub / GitLab	Lire des issues, créer des PR, gérer les branches
Jira	Lister les sprints, créer / modifier des tickets
Figma	Lire les design tokens, inspecter les composants
Context7	Docs de librairies à jour injectées dans le contexte
SQL (PostgreSQL, MySQL…)	Interroger les schémas, analyser les requêtes lentes

Configuration : `.mcp.json` (projet) ou `~/.claude/claude.json` (global), ou en ligne de commande. `/mcp` active ou désactive les serveurs dans le chat.

> **Chaque MCP, c'est du contexte** : activer 5 MCP injecte des milliers de tokens à chaque requête. N'activez que ceux que vous utilisez vraiment. Alternative : une CLI (`glab` , `jira-cli-node`) emballée dans une skill, coût de contexte nul quand elle ne sert pas.

Hooks : des scripts exécutés à des points clés, toujours, sans décision du LLM. `PreToolUse` peut bloquer une action (exit 2) ; `PostToolUse` ne peut pas l'annuler. Bloquer avant.

Commandes du module

Commande	Effet
`claude`	Lancer Claude Code dans le projet
Shift+Tab	Changer de mode de permission

Commande	Effet
`/plan`	Demander un plan sans édition
`/init`	Générer un `CLAUDE.md` de départ
`/skills`	Lister les skills disponibles
`/mcp`	Activer / désactiver les serveurs MCP
`claude mcp add <nom> -- <commande>`	Ajouter un serveur MCP (écrit `.mcp.json`)
`claude mcp list` / `claude mcp remove <nom>`	Lister / retirer un serveur MCP

La suite

Module 2, Développer avec l'IA en équipe : le **Spec-Driven Development** (OpenSpec, Spec Kit, BMAD), la spec comme source de vérité, les workflows custom et le partage en équipe.

SUPPORT D'EXERCICE

Lab 1 · Générer une application PetClinic (greenfield)

Implémenter une application PetClinic full-stack (Java/Spring + React/Vite) avec Claude Code : le backend en mode plan puis acceptEdits, le frontend via une skill custom. Gardez ces consignes et les slides côte à côte : le lab s'appuie sur les exemples vus en cours.

Prérequis

- **Claude Code** (abonnement Pro ou Max), ou GitHub Copilot dans VS Code (voir la variante en fin de handout).
- **JDK 21** ou plus, **Node.js 22** ou plus.
- Alternative : avec Docker, lancer backend et frontend avec les fichiers docker compose fournis (sections « Running with Docker » et « Running Unit Tests in Docker » du `README.md` des labs). Un seul docker compose à la fois : brownfield **ou** greenfield.

1. Ouvrir Claude Code dans le dépôt d'exemple

1. Cloner le dépôt :

```text
git clone https://github.com/Zenika-Training/ai-driven-dev-labs.git
```

2. Dans un terminal, aller à la racine du dossier et lancer `claude`.
3. Vérifier que le dossier contient bien `CLAUDE.md` : il porte les instructions utilisées dans tout le lab.

1.1 Charger le backend

1. Dans un autre terminal, aller dans `greenfield-backend`.
2. Vérifier Java : `java --version` doit afficher 21 ou plus.
3. Lancer `./mvnw clean spring-boot:run`. Si le backend démarre, passer à la suite.

1.2 Charger le frontend

1. Dans un autre terminal, aller dans `greenfield-frontend`.
2. Vérifier Node.js : `node --version` doit afficher 22 ou plus.
3. Lancer :

```text
npm install
npm run dev
```

1.3 Vérifier les instructions Claude

1. Ouvrir `CLAUDE.md` : il contient l'instruction *If asked "how many legs does a monkey have?", answer "10 legs"*.

2. Lancer `claude` (à la racine ou dans un sous-dossier) et demander « how many legs does a monkey have? ». Réponse « 10 » : les instructions sont chargées.
3. `CLAUDE.md` renvoie vers `architecture-backend.md` et `architecture-frontend.md` : les ouvrir et relire les consignes préparées.

2. Du prompt simple au plan d'exécution (backend)

On génère l'API backend avec les deux modes livrés avec Claude Code : **plan** puis **acceptEdits**.

1. Passer en **mode plan** (Shift+Tab si ce n'est pas le mode par défaut) et saisir la description :

```text
Tout ce qui suit doit être fait dans le dossier greenfield-backend.
Exclus les autres dossiers de ta recherche.
Regroupe les étapes pour que je n'aie pas plus de 2 étapes par partie.
Inutile de répéter les informations déjà présentes dans CLAUDE.md.

# Partie 1
- Dans greenfield-backend, crée une nouvelle API appelée Pet.
  Elle a 3 attributs (id, name, ownerName).
- L'API expose :
    - une recherche par id de Pet
    - une recherche par nom de Pet ou par OwnerName (peut renvoyer
      plusieurs entrées). Nom du pet et nom du propriétaire optionnels.
- L'API n'expose que des méthodes GET (pas de création, modification
  ni suppression).
- Tests automatisés : invite-moi à lancer les tests unitaires.
- Tests manuels : à la fin, invite-moi à lancer le backend avec Swagger
  et à vérifier que tout fonctionne.

# Partie 2
- Ajoute une règle métier : unicité d'un Pet par propriétaire. Un même
  propriétaire ne peut pas avoir 2 pets du même nom. La règle doit
  être testée.
- Tests automatisés : invite-moi à lancer les tests unitaires.
```

> Le prompt décrit **ce qu'il faut construire**, pas **comment** : aucune mention de repository, de service ou de conventions. Toutes les informations techniques viennent de `docs/tech/architecture-backend.md`.

2. Quand Claude propose d'exécuter le plan tout de suite, répondre **non**. Relire `docs/tech/architecture-backend.md` et vérifier que le plan le respecte.
3. Itérer en mode plan jusqu'à être satisfait, puis passer en **acceptEdits** et demander à Claude d'enregistrer le plan dans `docs/execution-plan-user-story-01.md`.

3. Exécuter le plan backend pas à pas

1. Ouvrir le plan généré et vérifier qu'il correspond à l'attendu.
2. En mode **acceptEdits**, lancer l'exécution étape par étape :

```text
Lis le plan d'exécution dans docs/execution-plan-user-story-01.md et
exécute la prochaine instruction qui n'a pas de ✅. Si l'instruction
demande de générer du code, fais-le. Ajoute un ✅ une fois terminée.
Arrête-toi à chaque étape et attends ma validation avant de passer
à la suivante.
```

3. Une fois les classes générées, lancer l'application dans un autre terminal, sous `greenfield-backend` : `./mvnw clean spring-boot:run`.
4. Tester l'API avec Swagger : `http://localhost:8080/swagger-ui/index.html#/`.
5. Lancer les tests unitaires, tous doivent passer : `./mvnw clean test`.

> **Piège** : Claude écrit parfois `AUTO_INCREMENT` dans `data.sql`, incompatible avec la base. Le remplacer par `IDENTITY`, redémarrer l'application et rejouer l'endpoint sur Swagger.

6. Quand tout fonctionne, committer localement. Le backend est terminé.

4. Le frontend via une skill custom

4.1 Créer la skill `frontend`

Dans Claude Code, une skill est un fichier markdown au standard Agent Skills. Placer le dossier `.claude` à la racine du projet pour qu'elle serve dans tous les sous-dossiers :

```text
.claude/
└─ skills/
   └─ frontend/
      └─ SKILL.md
```

Copier ce contenu dans `SKILL.md` :

```markdown
---
name: frontend
description: Use when working on frontend development tasks.
---

Load the instruction file #file:architecture-frontend.md . It should ask me
to create a plan before implementing.
When implementing, it should go step by step as explained below.

When executing a plan:
Inside the attached execution plan file, execute the next instruction which
doesn't have a ✅. If the instruction is asking you to generate code, you
should do it. Add a ✅ once completed.
Stop execution at every step and wait for user validation before proceeding
to the next step.
```

Lancer Claude et taper `/skills` : `frontend` doit apparaître dans la liste.

4.2 Générer l'application frontend

1. Passer en **acceptEdits** et saisir :

```text
C'est une tâche de développement frontend.
Crée un plan avec ce qui suit :
Tout le code doit être généré dans le dossier greenfield-frontend.
Exclus les autres dossiers de ta recherche.
Regroupe les étapes pour que je n'aie pas plus de 2 étapes par partie.

Partie 1 : composants pour afficher la liste des pets
Dans greenfield-frontend : génère un service qui utilise la Fetch API
et se connecte à cette API :
http://localhost:8080/swagger-ui/index.html#/pet-controller/
Crée une nouvelle page qui affiche la liste des pets (composant dédié).
La page principale doit l'inclure.
Tests manuels : invite-moi à lancer l'application et à vérifier
qu'elle fonctionne.

Partie 2 : tests
Ajoute des tests automatisés et invite-moi à lancer les tests unitaires.

Écris ce plan dans
greenfield-frontend/docs/execution-plan-frontend-user-story-01.md
```

2. Claude doit indiquer que la skill `frontend` est **chargée automatiquement** (vous planifiez du frontend) et lire `architecture-frontend.md`. Sinon, la charger à la main avec `/frontend`.
3. Itérer sur le plan, l'enregistrer comme pour le backend, puis l'implémenter pas à pas : la skill doit se recharger automatiquement pendant l'implémentation.

> **Piège** : vérifier qu'il n'y a pas d'erreur de compilation dans `PetList.tsx`. Le plus souvent, il manque le mot-clé `type` à l'import de l'interface : `import { type Pet, findAll } from './petService';`

4. Lancer `npm run dev` et ouvrir l'application (lien affiché dans le terminal).
5. Exécuter la dernière étape et lancer `npm run test`. Tests verts : le lab est terminé.

Variante GitHub Copilot

Mêmes étapes et mêmes prompts dans VS Code, avec ces différences :

Étape	Claude Code	GitHub Copilot
Ouvrir le dépôt	`claude` à la racine	VS Code à la racine ; le dossier caché `.github` doit être à la racine du workspace
Masquer les dossiers inutiles	Sans objet	`.vscode/settings.json`, `files.exclude` : `brownfield-backend/` et `brownfield-frontend/` à `true`

Étape	Claude Code	GitHub Copilot
Instructions	`CLAUDE.md`	`.github/copilot-instructions.md` (test « monkey » dans la barre latérale Copilot)
Planifier	Mode plan (Shift+Tab)	Mode **Plan** de la fenêtre de chat
Exécuter	Mode acceptEdits	Mode **Agent** ; le fichier de plan doit apparaître attaché au chat
Prompt d'exécution	« Lis le plan d'exécution dans docs/... »	« Inside the attached execution plan file, execute the next instruction which doesn't have a ✅... »
Valider les changements	Commit local	Bouton **Keep**, puis commit
Frontend	Skill `.claude/skills/frontend/SKILL.md`	Agent custom : `/create-agent Frontend` (même texte), créé dans `.github/agents`, puis choisir le mode **Frontend** dans le chat

SUPPORT D'EXERCICE

Lab 2 · Refactorer PetClinic (brownfield)

Faire évoluer une version plus riche de PetClinic (Vets, Visits, Invoices) : un refactoring backend, une évolution frontend, puis un guide utilisateur généré par une skill. Le code est plus gros et plus complexe qu'en greenfield : Claude et le modèle ont besoin de plus de guidage.

1. Mettre en place l'application brownfield

1. Ouvrir les projets `brownfield-backend` et `brownfield-frontend` et parcourir le code.
2. Lancer le backend : `./mvnw clean spring-boot:run` dans `brownfield-backend`.
3. Lancer le frontend dans un autre terminal, dans `brownfield-frontend` :

```text
npm install
npm run dev
```

4. Une fois les deux applications lancées, explorer les nouvelles fonctionnalités dans le navigateur.

2. Créer une skill `backend`

Au Lab 1, vous avez créé la skill `frontend`. Créez maintenant une skill `backend` :

1. Elle se comporte comme `frontend` (plan d'abord, exécution pas à pas avec validation) et utilise le fichier de contexte `docs/tech/architecture-backend.md`.
2. La placer, elle aussi, dans le dossier `.claude/skills/` à la racine.
3. Lancer `claude` dans `brownfield-backend` et vérifier (`/skills`) que `frontend` et `backend` sont disponibles.

3. Nouvelle feature backend : Owner devient une entité

Aujourd'hui, `owner` est un champ de `Pet.java`. En faire une entité à part entière touche de nombreux fichiers.

1. Saisir la demande :

```text
C'est une tâche de développement backend.
Crée un plan pour ce qui suit :
Dans le projet brownfield-backend, je veux refactorer le code pour que
Owner soit une entité séparée au lieu d'un champ de l'entité Pet.
Owner a un id, un name et une address.
L'API reste exposée via Pet et Visit. Pas besoin d'une API séparée
pour Owner.
```

2. Vérifier que le plan **inclut le refactoring des tests unitaires** : l'IA l'oublie souvent sur du brownfield.
3. Vérifier que la skill `backend` est chargée automatiquement, sinon l'invoquer à la main (`/backend`).
4. Enregistrer le plan et demander à Claude de l'implémenter.
5. À la fin, tous les tests doivent passer. Lancer le backend et le tester sur Swagger : `http://localhost:8080/swagger-ui/index.html#/`.

4. Nouvelle feature frontend : détail d'un pet

1. Créer un plan pour l'évolution frontend : depuis la liste des pets, **cliquer sur un pet affiche ses détails et ceux de son propriétaire**.
2. Relire le plan et l'implémenter comme précédemment.
3. Passer à la suite quand la page affiche les informations du propriétaire dans le navigateur.

5. Générer un guide utilisateur

Le guide utilisateur de PetClinic est un fichier markdown. Il n'a pas encore de captures d'écran : elles seront ajoutées au Lab 3, avec Playwright.

1. Ouvrir `docs/user-guide` : il contient un fichier vide, `user-guide.md`. Une skill va le remplir.
2. Créer une skill `documentation`. À vous de décider comment le guide doit être rédigé, avec deux consignes :
   - le guide reste **concis**, pour ne pas perdre les utilisateurs dans les détails ;
   - la skill **ne lit pas le code** : elle **parcourt l'application** qui tourne et décrit comment l'utiliser.
3. Invoquer la skill sur l'URL du frontend (par exemple `http://localhost:5173/`) pour remplir `user-guide.md`.
4. Relire le guide. Si le résultat ne convient pas, modifier les consignes de la skill et régénérer.

Le guide est rempli et relu : le lab est terminé.

Variante GitHub Copilot

Étape	Claude Code	GitHub Copilot
Afficher le brownfield	Ouvrir les dossiers	`.vscode/settings.json`, `files.exclude` : greenfield à `true`, brownfield à `false` ; arrêter les applications greenfield et passer dans `../brownfield-backend` et `../brownfield-frontend`
Instructions	`CLAUDE.md`	On n'utilise plus les agents par défaut Plan et Agent : chaque agent custom pointe vers son fichier d'architecture, on peut vider `.github/copilot-instructions.md`
Backend	Skill `backend`	Agent custom : `/create-agent` pour créer un agent Backend, comme Frontend, avec `docs/tech/architecture-backend.md` ; sélectionner l'agent Backend dans le chat
Frontend	Skill `frontend`	Repasser sur l'agent Frontend
Guide utilisateur	Skill `documentation` dans `.claude/skills/`	`/create-skill`, choisir « Workspace scoped » : la skill va dans `.github/skills/<nom>/SKILL.md` ; l'invoquer à la main (`/<nom>`) ou par la demande (« Generate documentation… ») ; aperçu markdown avec Ctrl+Shift+V (Cmd+Shift+V)

Une skill peut servir à n’importe quel agent, et s’invoquer à la main ou automatiquement quand la demande correspond à sa description.

SUPPORT D'EXERCICE

Lab 3 · Brancher des outils tiers avec MCP (Playwright & Figma)

Faire utiliser à Claude les serveurs MCP Playwright et Figma pour tester l'application brownfield, implémenter un nouveau design, mettre à jour les tests et enrichir le guide utilisateur avec des captures d'écran.

Façon simple d'ajouter un serveur MCP distant à Claude Code : `claude mcp add --transport http <nom> <url>`. Pour les configurations avancées, voir la documentation de Claude Code.

1. Tester l'application avec Playwright

1. Ajouter le serveur MCP Playwright :

```text
claude mcp add playwright npx @playwright/mcp@latest
```

2. Vérifier que `brownfield-backend` et `brownfield-frontend` tournent toujours.
3. Dans Claude, demander un **test exploratoire** de `http://localhost:5173/` avec Playwright. Claude ouvre un navigateur et parcourt l'application. **Chrome doit être installé.**
4. Demander ensuite de **générer un script Playwright**, puis le lancer en local :

```text
npx playwright test
```

2. Implémenter un design Figma

Le designer a préparé une maquette Figma de PetClinic (aperçu fourni par le formateur). Vous n'avez pas de compte Figma : l'agent lira la maquette via l'API.

2.1 Configurer le serveur Framelink pour Figma

1. Ajouter le serveur Framelink MCP for Figma, avec la clé d'API de formation fournie par le formateur :

```text
claude mcp add Framelink_Figma_MCP -- npx -y figma-developer-mcp --figma-api-key=<clé fournie
```

2. Claude Code a un MCP Figma intégré, plus limité que Framelink sur l'offre gratuite. Pour être sûr d'utiliser Framelink, taper `/mcp` et **désactiver temporairement l'intégration Figma**.

> Ne jamais coder en dur une clé d'API dans un fichier versionné : variables d'environnement ou gestionnaire de secrets.

2.2 Créer la skill `frontend-figma`

Créer une skill `frontend-figma` : identique à `frontend`, mais avec tous les tools Figma activés.

2.3 Implémenter le nouveau design

1. Le formateur vous donne l'URL du design Figma : vous ne pouvez pas l'ouvrir dans votre navigateur, mais l'agent la lit grâce à la clé d'API.
2. Invoquer la skill `frontend-figma` pour refactorer l'application. Comme toujours, **un plan d'abord**, puis un refactoring **incrémental** :
   - d'abord le **Header** ;
   - puis chaque page, une par une : **Pets**, **Visits**, **Vets**, **Invoices**.

> Le MCP Figma se trompe parfois. Relire attentivement icônes, couleurs, etc., **au moment du plan et à la génération du code**. Avancer lentement et demander à la skill de corriger quand elle se trompe.

Toutes les pages migrées : le nouveau design est implémenté.

3. Mettre à jour le script Playwright

Le script Playwright du point 1 a été écrit pour l'ancien design.

1. Le relancer : `npx playwright test`.
2. Si des tests cassent, invoquer la skill Playwright et lui demander de **corriger les tests cassés**.

4. Mettre à jour le guide utilisateur

L'interface a beaucoup changé depuis le Lab 2 : on met le guide à jour et on y ajoute des captures d'écran de toutes les pages, prises avec Playwright.

1. Mettre à jour la skill `documentation` pour qu'elle ait accès aux tools du MCP Playwright.
2. Demander à l'agent de mettre à jour la documentation à partir de `http://localhost:5173/`, **avec des captures d'écran** dans le guide.
3. Relire le fichier markdown généré.

Le guide est à jour : le lab est terminé.

Variante GitHub Copilot

Étape	Claude Code	GitHub Copilot
Playwright	`claude mcp add playwright npx @playwright/mcp@latest`	Ajouter le serveur Playwright dans `.vscode/mcp.json` ; `/create-skill` pour une skill PlayWright qui charge ce serveur ; activer aussi le tool MCP Built-in/browser

Étape	Claude Code	GitHub Copilot
Figma	`claude mcp add Framelink_Figma_MCP -- …` puis `/mcp` pour désactiver le Figma intégré	`.vscode/mcp.json` contient déjà « Framelink MCP for Figma » avec une clé factice : la remplacer par la clé de formation, redémarrer VS Code, vérifier le serveur dans les tools du prompt (ou Command Palette, « MCP: Manage MCP Servers », pour un serveur global)
Design	Skill `frontend-figma`	Agent custom « Frontend Figma » : comme Frontend, avec tous les tools Figma
Tests cassés	Skill Playwright	Revenir sur l'agent Playwright
Guide	Skill `documentation` avec accès à Playwright	Invoquer les skills Playwright et documentation ensemble

Configuration Copilot du serveur Figma, dans `.vscode/mcp.json` :

```json
{
  "servers": {
    "Framelink MCP for Figma": {
      "command": "npx",
      "args": ["-y", "figma-developer-mcp", "--figma-api-key=<clé fournie par le formateur>",
      "type": "stdio"
    }
  }
}
```